

CMSE 801 - Day 13



Using Pandas to make an informed assessment
based on data

Exploring the Flint water crisis data

October 9, 2024

General Course Announcements

- Quiz 1 grading continues! Homework 1 grades and grades through PCA 12 are up to date.
- Homework 2 is due **this Friday**, October 11, by 11:59pm.
- Quiz 2 will take place on **Monday** (October 14).
 - Some folks indicated a preference for taking the quiz right away at the beginning of class. I'm going to send out a quick poll to get your preference and we'll go based on the majority.
 - The content of the quiz will cover concepts and assess knowledge that you demonstrated in Homework 2.
- Homework 3 has been posted and is due **October 25**.

Notes about homework submissions

- Make sure to read the assignment directions. Some lost points were simply a result of not doing what the question asked for or not doing all of the questions.
- Using a generative AI is perfectly acceptable in the course, but remember to include details about what tool you used and how you used it when you do (this include HiTA).
- Before you submit the file, you should make sure you do a "Kernel" -> "Restart and Clear Output" and then then re-run every cell in the note (in order) to make sure everything still works. Submit your notebook with all of the cells executed to simplify the grading process.

Questions/comments from Day 13 PCA

- Most common challenges were:
 - Masking and how it works -- **we'll look at that!**
 - Making subplots. These are use for seeing multiple plots from one code cell, which can be really useful, but not the only way we can do this. **We'll look at methods with and without using `plt.subplot()`.**
 - Using the logarithmic scale on plots, **we'll review this** and look at more than one option for making log plots as well.

Before we march on, let's import the modules we might use.

```
In [1]: import numpy as np  
import matplotlib.pyplot as plt  
import pandas as pd
```

Let's do a little bit of experimentation with masks

```
In [7]: # Let's generate some fake data  
my_data = np.random.randint(low=0, high=100, size=100)  
print(my_data)
```

```
[83 99 56  1 88 77 89 71 66 44 78 88  9 30 54 10 48 88 12 15 49  
61 30  7  
24 54 98 69 28 43 75 90 19 56 85 76  3 24 16 87 49 30 73 35 30  
82 69 48  
13 33 78  2 85 73 63 43 50 63 63 91 27  2 16 69 41 10 52 14  5  
25 63 89  
47 63 73 66 37 99 42 63 63 79 85 63 17 41 21 57 68 22 91 81 45  
8 90 56  
1 77 90 66]
```

```
In [8]: mask = my_data > 25  
#print(mask)  
print(my_data[mask])  
print("Number of values greater than 25:", len(my_data[mask]))
```

```
[83 99 56 88 77 89 71 66 44 78 88 30 54 48 88 49 61 30 54 98 69
28 43 75
 90 56 85 76 87 49 30 73 35 30 82 69 48 33 78 85 73 63 43 50 63
63 91 27
 69 41 52 63 89 47 63 73 66 37 99 42 63 63 79 85 63 41 57 68 91
81 45 90
 56 77 90 66]
```

Number of values greater than 25: 76

```
In [10]: mask = my_data == 25
print(mask)
print(my_data[mask])
print("Number of values equal to 25:", len(my_data[mask]))
```

```
[False False False False False False False False False False False False
False False
False False False False False False False False False False False False
False False
False False False False False False False False False False False False
False False
False False False False False False False False False False True False
False False
False False False False False False False False False False False False
False False
False False False False]
[25]
Number of values equal to 25: 1
```



```
In [11]: mask1 = my_data > 25
mask1_data = my_data[mask1]
mask2 = mask1_data < 75
mask1and2_data = mask1_data[mask2]
print(mask1and2_data)
print("Number of values between 25 and 75:", len(mask1and2_data))
```

```
[56 71 66 44 30 54 48 49 61 30 54 69 28 43 56 49 30 73 35 30 69
48 33 73
 63 43 50 63 63 27 69 41 52 63 47 63 73 66 37 42 63 63 63 41 57
68 45 56
66]
```

```
Number of values between 25 and 75: 49
```

```
In [12]: mask_both = (my_data > 25) & (my_data < 75) # this uses a bitwise "AND"
masked_data = my_data[mask_both]
print(masked_data)
print("Number of values between 25 and 75:", len(masked_data))
```

```
[56 71 66 44 30 54 48 49 61 30 54 69 28 43 56 49 30 73 35 30 69
 48 33 73
 63 43 50 63 63 27 69 41 52 63 47 63 73 66 37 42 63 63 63 41 57
 68 45 56
 66]
Number of values between 25 and 75: 49
```

This is using something we haven't seen/talked about, which are ["bitwise" operators](#) in Python.

```
In [13]: mask_both = (my_data < 25) | (my_data > 75) # this use a bitwise "OR"
masked_data = my_data[mask_both]
print(masked_data)
print("Number of values less 25 and greater 75:", len(masked_data))
```

```
[83 99  1 88 77 89 78 88  9 10 88 12 15  7 24 98 90 19 85 76  3
24 16 87
 82 13 78  2 85 91  2 16 10 14  5 89 99 79 85 17 21 22 91 81  8
90  1 77
 90]
Number of values less 25 and greater 75: 49
```

We can also be a bit lazier and save ourselves some lines of code...

```
In [14]: print(my_data[my_data>25]) # the mask I was using before is directly placed  
print("Number of values greater than 25:", len(my_data[my_data>25]))
```

```
[83 99 56 88 77 89 71 66 44 78 88 30 54 48 88 49 61 30 54 98 69  
28 43 75  
90 56 85 76 87 49 30 73 35 30 82 69 48 33 78 85 73 63 43 50 63  
63 91 27  
69 41 52 63 89 47 63 73 66 37 99 42 63 63 79 85 63 41 57 68 91  
81 45 90  
56 77 90 66]  
Number of values greater than 25: 76
```

We can using masks with Pandas data as well!

```
In [15]: from io import StringIO
# read in some data on heart disease
heart_disease_data = pd.read_json(StringIO("""[{"observation_number":0,'
heart_disease_data.head()
```

```
Out[15]:
```

	observation_number	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exa
0	0	67	1	4	160	286	0	2	108	
1	1	67	1	4	120	229	0	2	129	
2	2	37	1	3	130	250	0	0	187	
3	3	41	0	2	130	204	0	2	172	
4	4	56	1	2	120	236	0	0	178	

```
In [16]: over_60_mask = heart_disease_data['age'] > 60
over_60_data = heart_disease_data[over_60_mask]
over_60_data.head(10)
```

Out[16]:

	observation_number	age	sex	cp	trestbps	chol	fbs	restecg	thalach	e
0	0	67	1	4	160	286	0	2	108	
1	1	67	1	4	120	229	0	2	129	
5	5	62	0	4	140	268	0	2	160	
7	7	63	1	4	130	254	0	2	147	
19	19	64	1	1	110	211	0	2	144	
26	26	66	0	1	150	226	0	0	114	
29	29	69	0	1	140	239	0	0	151	
31	31	64	1	3	140	335	0	0	158	
38	38	61	1	3	150	243	1	0	137	
39	39	65	0	4	150	225	0	2	114	

```
In [17]: # Again, I can be lazier about this  
over_60_data = heart_disease_data[heart_disease_data['age'] > 60]  
over_60_data.head(10)
```

```
Out[17]:
```

	observation_number	age	sex	cp	trestbps	chol	fbs	restecg	thalach	e
0	0	67	1	4	160	286	0	2	108	
1	1	67	1	4	120	229	0	2	129	
5	5	62	0	4	140	268	0	2	160	
7	7	63	1	4	130	254	0	2	147	
19	19	64	1	1	110	211	0	2	144	
26	26	66	0	1	150	226	0	0	114	
29	29	69	0	1	140	239	0	0	151	
31	31	64	1	3	140	335	0	0	158	
38	38	61	1	3	150	243	1	0	137	
39	39	65	0	4	150	225	0	2	114	

Let's review some of the plotting components well

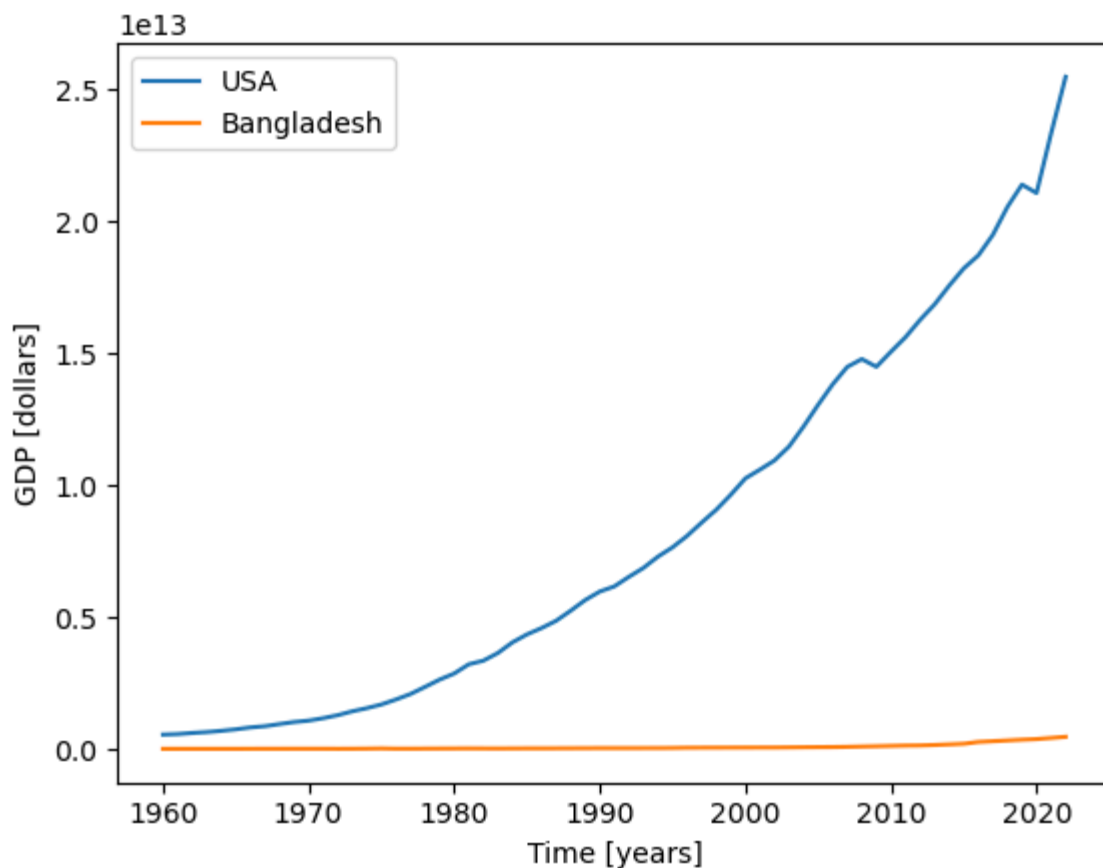
```
In [18]: # First we need to load some data to work with
gdp = pd.read_csv("GDP_Cleaned.csv")
gdp.head()
```

```
Out[18]:
```

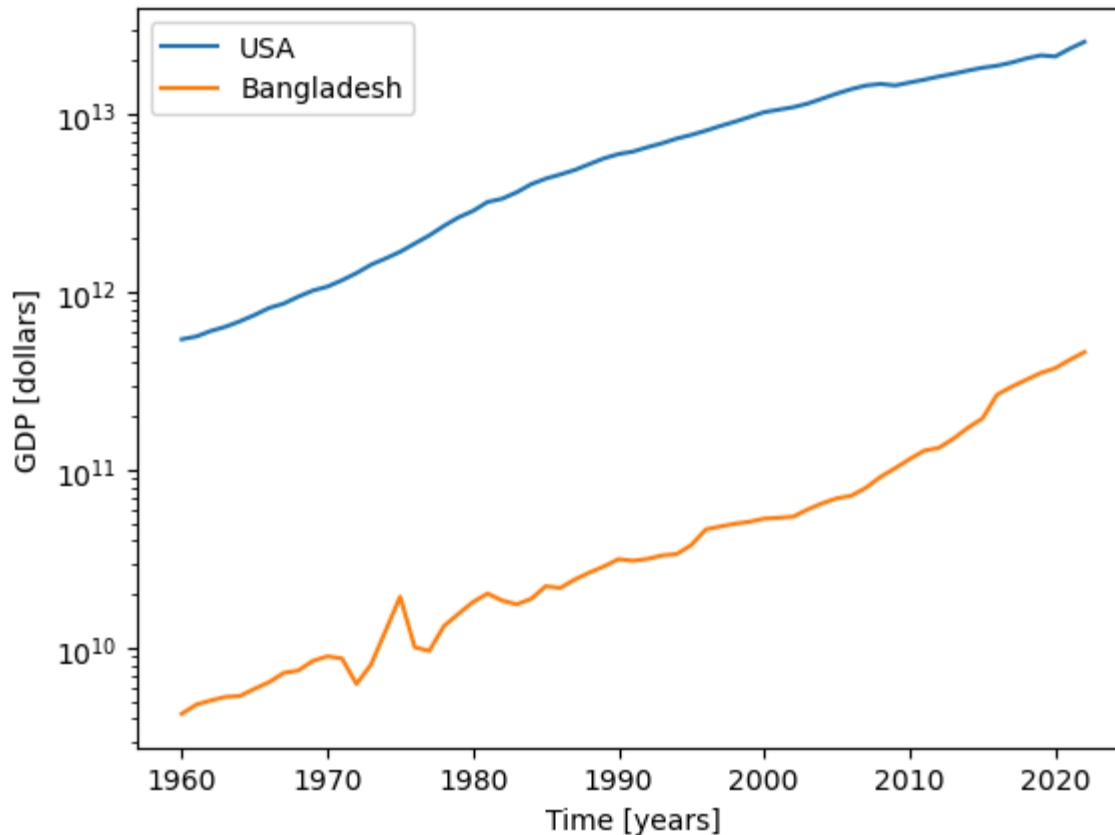
	Year	Africa Eastern and Southern	Africa Western and Central	Australia	Austria	Burun
0	1960	2.112502e+10	1.044764e+10	1.860656e+10	6.592694e+09	1.960000e+09
1	1961	2.161623e+10	1.117321e+10	1.968288e+10	7.311750e+09	2.030000e+09
2	1962	2.350628e+10	1.199053e+10	1.992256e+10	7.756110e+09	2.135000e+09
3	1963	2.804836e+10	1.272769e+10	2.153984e+10	8.374175e+09	2.327500e+09
4	1964	2.592067e+10	1.389811e+10	2.380112e+10	9.169984e+09	2.607500e+09

5 rows × 123 columns

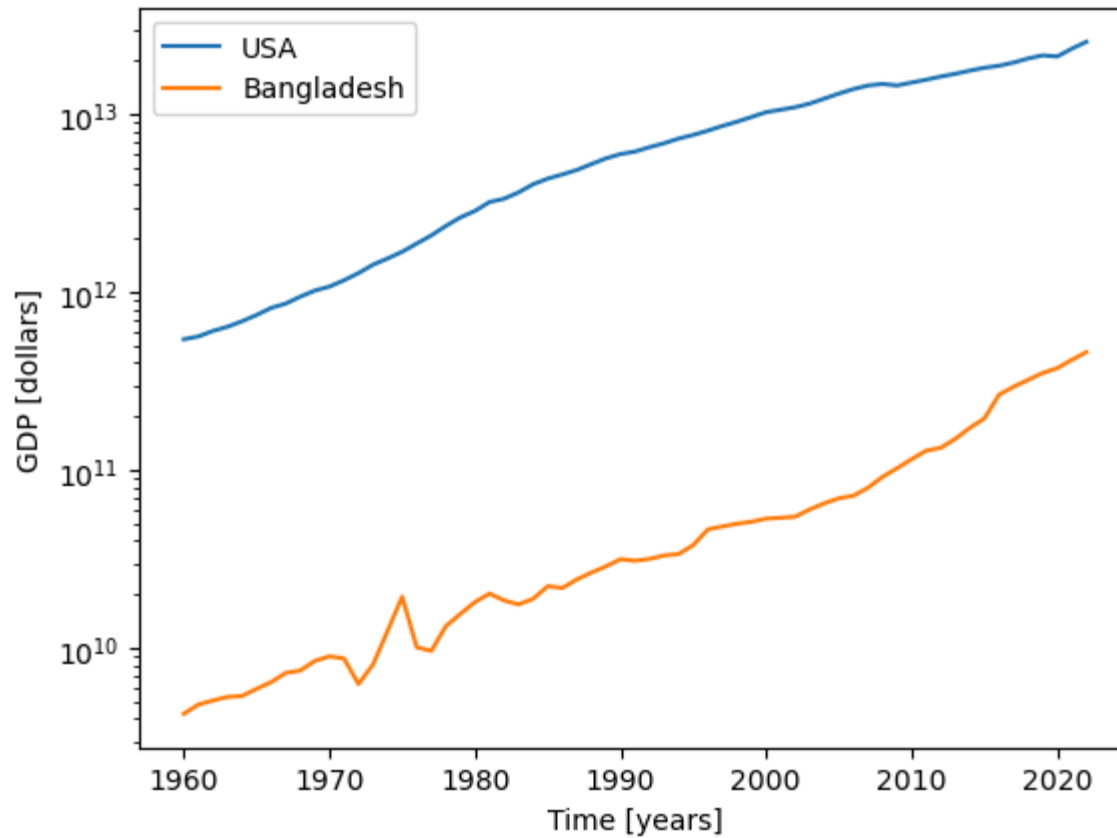
```
In [19]: plt.plot(gdp["Year"],gdp['United States'],label="USA")
plt.plot(gdp["Year"],gdp['Bangladesh'], label="Bangladesh")
plt.xlabel("Time [years]")
plt.ylabel("GDP [dollars]")
plt.legend();
```



```
In [24]: plt.plot(gdp["Year"],gdp['United States'],label="USA")
plt.plot(gdp["Year"],gdp['Bangladesh'], label="Bangladesh")
plt.yscale("log")
plt.xlabel("Time [years]")
plt.ylabel("GDP [dollars]")
plt.legend();
```



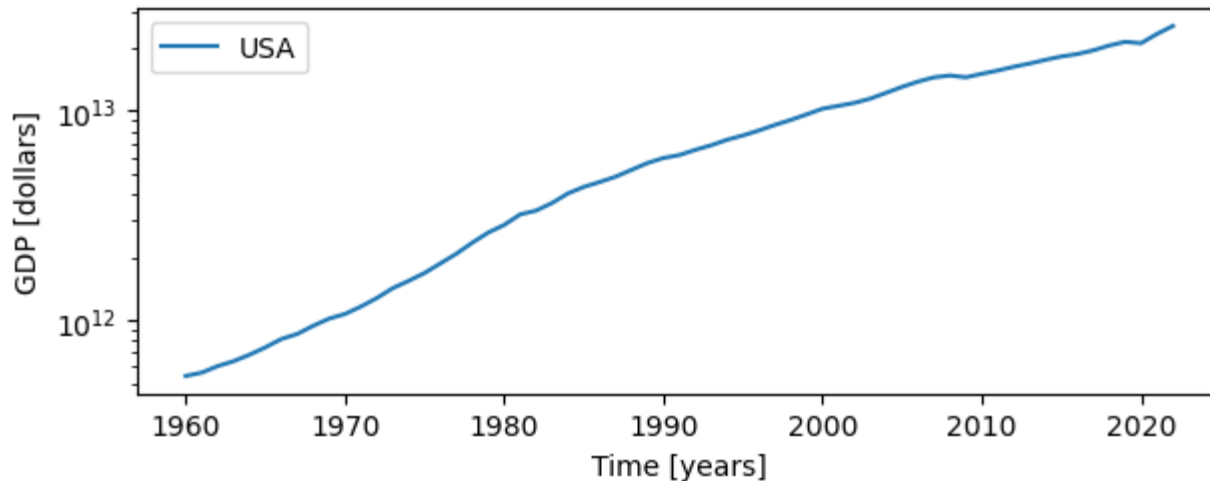
```
In [25]: plt.semilogy(gdp["Year"],gdp['United States'],label="USA")
plt.semilogy(gdp["Year"],gdp['Bangladesh'], label="Bangladesh")
plt.xlabel("Time [years]")
plt.ylabel("GDP [dollars]")
plt.legend();
```

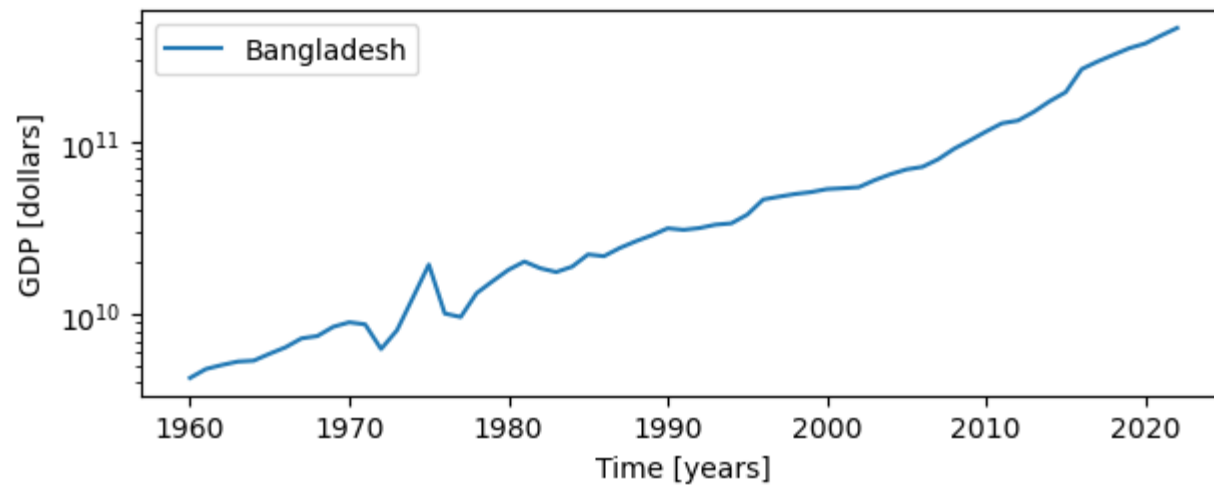


What if I wanted separate plots from one cell?

```
In [28]: plt.figure(1, figsize=(7,2.5))
plt.semilogy(gdp["Year"],gdp['United States'],label="USA")
plt.xlabel("Time [years]")
plt.ylabel("GDP [dollars]")
plt.legend();

plt.figure(2, figsize=(7,2.5))
plt.semilogy(gdp["Year"],gdp['Bangladesh'], label="Bangladesh")
plt.xlabel("Time [years]")
plt.ylabel("GDP [dollars]")
plt.legend();
```



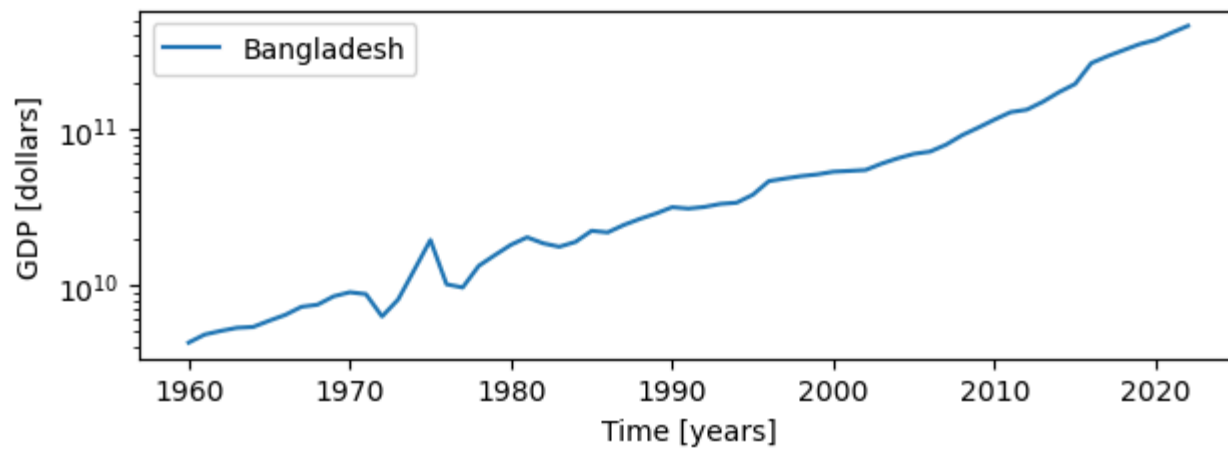
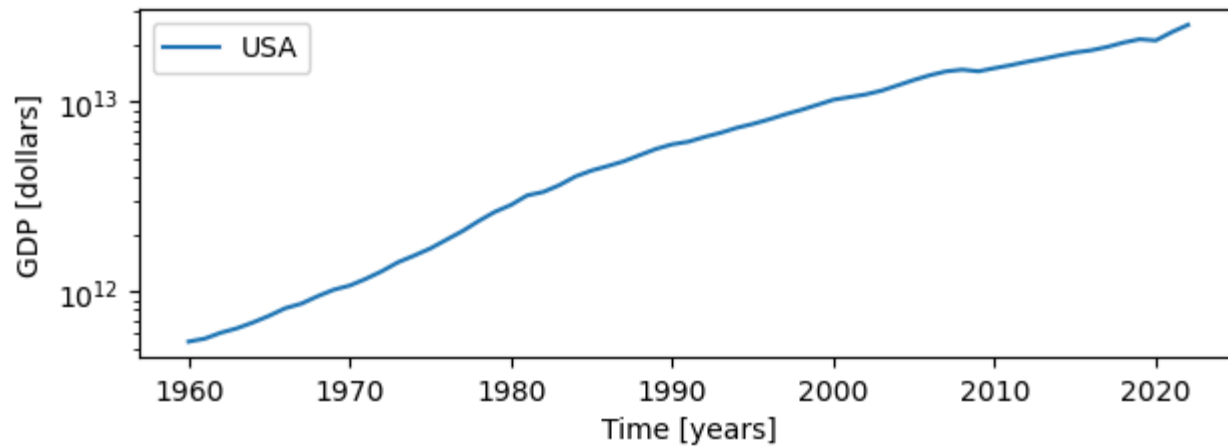


Or with plt.subplot()

```
In [31]: plt.figure(1)
plt.subplot(211)
plt.semilogy(gdp["Year"],gdp['United States'],label="USA")
plt.xlabel("Time [years]")
plt.ylabel("GDP [dollars]")
plt.legend();

plt.subplot(212)
plt.semilogy(gdp["Year"],gdp['Bangladesh'], label="Bangladesh")
plt.xlabel("Time [years]")
plt.ylabel("GDP [dollars]")
plt.legend();

plt.tight_layout();
```



Goals for today

- Load and explore data from the Flint water crisis using **Pandas**.
- By analyzing and visualizing the data in a variety of ways, make an assessment as to whether the lead levels in the Flint water supplied violated the EPA guidelines.

Looking forward - Day 14 PCA: Fitting models, making predictions, evaluating fits

- You'll spend some more time thinking about what it means to "fit a model" to data and try to devise your own algorithm for calculating a "goodness of fit"
- You'll also explore the `curve_fit` tool from SciPy, which can be used to fit models to data.

